STUDENT RESULT MANAGEMENT SYSTEM

Submitted by:

Abdullah Nasir (CT-25076)

M. Saad Ahmed (CT-25089)

Faez Ur Rehman (CT-25090)

INSTRUCTOR: Sir Abdullah

INDEX:

UNDERSTANDING AND USAGE OF CODE:

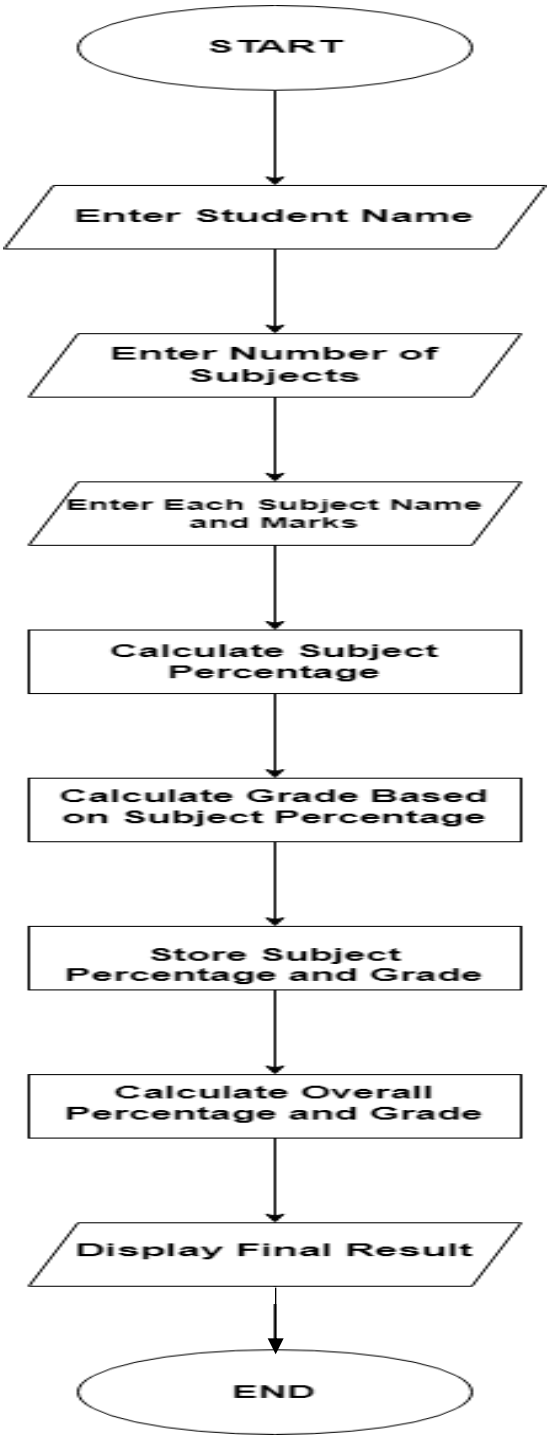
This C program is a simple **Student Result Management System** that takes a student's name, the number of subjects,

and the marks of each subject, and then calculates the result along with grades. The program begins with a function named getGrade () which takes the percentage as input and returns a grade letter according to standard grading criteria (A for 80% and above, B for 70% and above, and so on). In the main () function, the program first asks the user to enter the student's name and the total number of subjects. Then, for each subject, the user enters the subject name and marks out of 100. These marks are stored in arrays and also summed up to calculate the total obtained marks. After this, the program prints the result in a table format where each subject's marks, percentage, and grade are displayed. The program then calculates the **overall percentage** by dividing the total obtained marks by the total possible marks (number of subjects × 100). Based on this percentage, the final overall grade is determined using the same getGrade () function. Finally, the program displays the student's total marks, overall percentage, and overall grade. This program is useful for understanding the use of **arrays, loops, functions, conditional statements, formatted output, and string input** in C programming, and can be used for school or college result-related mini projects.

IMPROVEMENT OF CODE:

The improvement of this code makes it more safe, organized, and practical by adding better input handling, validation, formatting, and file handling. Instead of using unsafe input methods, the name is now taken using fgets () which correctly reads full names including spaces, and marks are checked with a validation loop so the user cannot enter marks less than 0 or more than 100. The output table is formatted neatly so the result looks clear and professional. Along with this, file handling can also be added where a text file (like **result.txt**) is opened using fopen (), and the student's result (subject names, marks, percentages, and overall grade) is saved permanently using fprintf (). This makes the result reusable even after the program ends, similar to real school result systems. Finally, the file is closed using fclose () to ensure proper saving. In this way, the overall code becomes more reliable, user-friendly, and practical for real-world use.

FLOWCHART OF CODE:



OUTPUT OF CODE:

```
Enter Student Name:Hassan Khan
Enter the number of Subjects:4
Enter Name of Subject 1:Maths
Enter Marks in Maths(out of 100):85
Enter Name of Subject 2:English
Enter Marks in English(out of 100):72
Enter Name of Subject 3:Computer
Enter Marks in Computer(out of 100):91
Enter Name of Subject 4:Physics
Enter Marks in Physics(out of 100):86

========STUDENT RESULT=========
Student Name:'Hassan Khan'
-------------------------------------
Subject          Marks    Percentage      Grade
Maths            85.00    85.00%          A
English          72.00    72.00%          B
Computer         91.00    91.00%          A
Physics          86.00    86.00%          A
-------------------------------------
Total obtained marks:334.00
Total subject marks:400.00
overall percentage:83.50%
overall grade:'A'
-------------------------------------
```

DRY RUN TABLE OF CODE:

Name of Student: Hassan Khan

Enter Number of Subjects: 4

Step	Subject	Marks	Action / Result
1	English	85	Percentage = 85%, Grade = A
2	Math	72	Percentage = 72%, Grade = B
3	Computer	91	Percentage = 91%, Grade = A
4	Physics	86	Percentage = 86%, Grade = A
5	-	-	Total = 334, Overall % = 83.5%
6	-	-	Overall Grade = A

CONCLUSION:

In conclusion, this program provides a simple and effective way to manage a student's academic result by taking their name, number of subjects, and marks, then calculating each subject’s percentage and grade along with the overall performance. It demonstrates the use of arrays, loops, functions, and conditional statements in C to organize data and make decisions. The program also outputs results in a neat and readable format, making it easy to understand the student’s performance. Overall, this code is a useful example of how basic programming concepts can be combined to solve real-life problems such as generating student report records.